Project 2 on Neural networks

Tim Herrmann Holmen

FYS-STK4155
Department of Physics

This project ...

CONTENTS

I. Introduction 2
A. Motivation 2
B. What i have done 2
1. Regression problem 2
2. Classification problem 2
C. In this report 2

II. Theory 2
A. Cost functions and their gradients 2
1. MSE 2
2. Binary cross entropy: log loss 2
3. Multiclass cross entropy cost function 3
B. Activation functions and their gradients 3
1. Sigmoid logit function 3
2. RELU function 3

III. Methods 3
A. The Data: Regression 3
1. Reshaping the data 4
B. The algorithm 4
1. Backpropagation 4
C. Stochastic Gradient Descent 5
D. Neuralnetregressor 5
E. Implementation and analysis of Regressor 5
1. Model verification 5
F. Model selection 6
1. Skicit Learn 6
2. Cross Validation 6
3. GridSearchCV 6
4. OLS, Ridge and Lasso 7
G. Final test 7
H. Neuralnetclassifier 7
I. Neuralnetclassifier 7
J. Measure of quality 7
1. Confusion matrix 7

IV. Results 7

V. Discussion 7

VI. Conclusion 7

A. Use of AI/LLM Tools 7
1. Tools used 7
2. Text contributions 7
3. Code contributions 7
a. LLM Declaration this section: 7

References 8

I. INTRODUCTION

A. Motivation

Aritficial neural networks have shown great results and improvements in the last years, impacting fields such as science, poltics and philosophy. Large language models, image detection, drones and autonomous systems, all have variations of artificial neural networks as its basis. In politics we can hear about intelligent machines, aritificial intelligence and super intelligence. The motivation for this project is to learn about neural networks, demistify its supernatural appearance, and understand the mechanics and its capacity to learn and solve known functions and data relations. The method and results will be compared to known methods of linear regression and classification. As a test of strength and calibration.

B. What i have done

1. Regression problem

We have created a feed forward neural network; FFNN, used stochastic gradient descent; SGD, to find the minimum of the cost function, which is the MSE for the regression problem. The gradients are calculated using backpropagation, which uses the kernal rule of derivation on the FFNN in reverse; from the output and target MSE derivative to the derivative of the parameters of the first layer. The gradients are compared with jax, automatic differentiation, to ensure agreement. In the SGD the descent method used on the parameters and gradients is the Plain gradient descent; regular updating, RMSprop; complex updating, Adam; best updating. Different layer width and depths are explored, along with the activation functions in the hidden layers, different learning rates are explored and compared. The output layer has the identity as activaiton function. The choice of activation function is the identity since the problem of choice is a regression problem. This network is compared with OLS for the one dimensional runge function. Further L1 and L2 penalties are applied to the cost function, with a variable penalty value λ, and these models are compared with Lasso and Ridge.

2. Classification problem

For classification, the network uses the softmax function in its output layer and cross-entropy as its cost function. These choices, together with the dataset, distinguish this network from the regression network. The task is multiclass classification of handwritten digits from MNIST; see `https://www.tensorflow.org/datasets/catalog/mnist`. We investigate the hidden-layer width, learning rate γ, and the plain, RMSprop, and Adam update methods. We also apply L1 and L2 penalties with different values of the regularisation parameter λ.

C. In this report

II III IV V VI The full project can be found at (TimH-Holmen, 2026a)

II. THEORY

A. Cost functions and their gradients

The loss function is the most important function for training, since training a model means to minimize the cost function of the model. To minimize the cost function, we use backpropagation to find the gradient of the cost, with regards to and gradient descent.

1. MSE

When we have our calculated output value $\tilde{y}$ and the true output value y, with n data points, the mean-squared error and its derivative is

$$MSE(y,\tilde{y}) = \frac{1}{n}\sum_{i=0}^{n-1}(y-\tilde{y})^2 \quad \frac{1}{n}||y - X\Theta||_2^2 \tag{1}$$

a. L2 norm which corresponds to the Ridge cost function.

$$MSE + \lambda||\Theta||_2^2 \quad \Theta(W,b) \tag{2}$$

b. L1 norm which corresponds to the Lasso cost function.

$$MSE + \lambda||\Theta|| \quad \Theta(W,b) \tag{3}$$

(Hjorth-Jensen, 2016, p. 86-91)

2. Binary cross entropy: log loss

With two classes, we assume a binary output $y \in \{0,1\}$, and a predicted probability $p \in [0,1]$ which is the output of the sigmoid function of z_i. Where $z_i = x_i^T\Theta$ is the linear combination of the input features and the model parameters Θ. For a single example, data point i,

the binary cross entropy cost function is defined as

$$C(\Theta) = -\sum_{i=1}^{n}(y_i \log p_i + (1-y_i)\log[1-p_i]) \quad (4)$$

$$p_i = \sigma(z_i) = \frac{1}{1+\exp(-z_i)} \quad (5)$$

$$(6)$$

The cost function is derived from the maximum likelihood estimation of the Bernoulli distribution, the negative log likelihood of the Bernoulli distribution. This is known as the cross entropy, and it is the loss function used to train binary classification models. With the linear function $z_i = x_i^T\Theta$, the gradient of the cost function with respect to the model parameters Θ is given by

$$\frac{dC(\theta)}{d\theta} = -X^T(y-p) \quad (7)$$

For N data points, the mean binary cross entropy is defines as

$$\frac{1}{N}\sum_{i=1}^{N} C(\theta)_i$$

And the term $1/N$ follows through te calculations to give the mean binary cross entropy gradient, which is given by

$$\nabla_\Theta C(\Theta) = -\frac{1}{n}X^T(y-p) \quad (8)$$

The cross entropy L1 and L2 norm is supplemented like the ordinary MSE.

3. Multiclass cross entropy cost function

For k klasses, one uses the soft max function to calculate the probability of each class. Again we have the linear function $z_k = X^T\Theta_k$ for each class, over all features and data points, the softmax functions is defined as

$$p(C=k \mid x) = \frac{\exp(z_k)}{\sum_{l=0}^{K-1}\exp(z_l)} k = 0, \ldots, K-1 \quad (9)$$

The nominator is for class k, and the denominator is the sum of all classes. The multiclass cross entropy cost function is defined as

$$C(\Theta) = -\sum_{i=1}^{n}\sum_{k=0}^{K-1} y_{ik}\log p(C=k|x_i) \quad (10)$$

Which is summed over all data points and classes. The gradient of the multiclass cross entropy cost function is given by

$$\frac{dC}{d\Theta_k} = -X^T(y_k - p_k) \quad (11)$$

The cross entropy L1 and L2 norm is supplemented like the ordinary MSE. Stability: The softmax function can be numerically unstable for large values of z_k, for a large value the e exponential function can overflow. Since the softmax is invariant udner subtracting a constant frome every score, one subtracts the largest before exponentiating:

$$p_k = \frac{\exp(z_k - z_{max})}{\sum_l \exp(z_l - z_{max})}$$

, $z_k = x^T\Theta_k$. (Hjorth-Jensen, 2016, p. 151-156)

B. Activation functions and their gradients

1. Sigmoid logit function

$$\sigma(z) = \frac{1}{1+\exp(-z)} \quad (12)$$

$$\sigma'(z) = \sigma(z)(1-\sigma(z)) \quad (13)$$

2. RELU function

$$ReLU(z) = \max(0, z) \quad (14)$$

$$ReLU'(z) = \begin{cases} 0 & z < 0 \\ 1 & z > 0 \end{cases} \quad (15)$$

(Hjorth-Jensen, 2016, p. 214-242)

III. METHODS

A. The Data: Regression

In this project, the input data is normally distributed, and consist of 100 data points. The target data is generated by Runge's function, with added noise, Runge Function which is a one-dimensional function of second order.

$$f(x) = \frac{1}{1+25x^2}$$

The noise is normally distributed with a mean of 0 and a standard deviation of 0.1.

1. Reshaping the data

The input and output data are reshaped into column vectors, (100,1), this is useful since most machine learning tools expect input features as a 2D array. (OpenAI, ????) For linear regression problems, the input data is expressed in terms of polynomial features, which is done by creating the design matrix of variable degree with the input data x, and adjusting the features during training, this is equal for ordinary least squares, lasso and ridge. For the neural network this is not necessary, since the network architecture, the number of nodes and layers, replaces the polynomial features, and the weights and biases are adjusted during training.

B. The algorithm

The set-up.

1. Fix the architecture: the number of inputs and outputs, the number of hidden layers and the number of nodes in each.
2. Choose activation functions for the hidden layers and for the output layer, the latter dictated by the range of the target.
3. Choose the cost function, again dictated by the distribution of the target, and any regularisation terms with their hyperparameters
4. Choose the optimiser - plain gradient dsecent, RMSprop, adam - and its learning rate.
5. Initialise the weights and biases

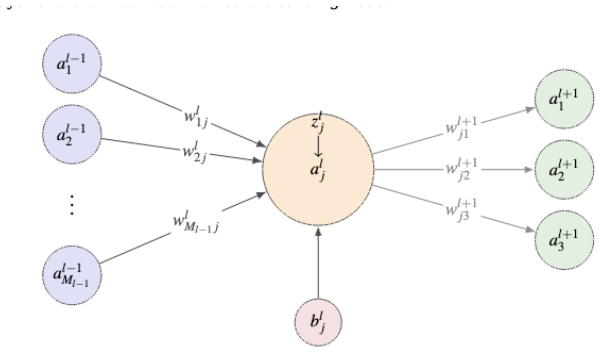


FIG. 1 The index convention for the equations below, seen from a single node $a_j^{(l)}$. The node j in layer l collects the outputs $a_i^{(l-1)}$ of all M_{l-1} nodes in the previous layer, weighted by $w_{ij}^{(l)}$, adds its own bias $b_j^{(l)}$ to form the pre-activation $z_j^{(l)}$, and applies its activation function σ to obtain $a_j^{(l)}$, which it then sends on to every node of layer $l+1$ through the weights $w_{jk}^{(l+1)}$.

For a single node, the pre-activation and activation are

$$z_j^{(l)} = \sum_{i=1}^{M_{l-1}} w_{ij}^{(l)} a_i^{(l-1)} + b_j^{(l)},$$

$$a_j^{(l)} = \sigma\left(z_j^{(l)}\right).$$

(Hjorth-Jensen, 2016, p. 326–328)

Each iteration:

1. *Forward pass.* Set $a^{(0)} = x$ and compute $z^{(l)}$ and $a^{(l)} = \sigma(z^{(l)})$ for $l = 1, 2, \ldots, L$, storing every $z^{(l)}$ and $a^{(l)}$:

$$z^{(l)} = a^{(l-1)} W^{(l)} + b^{(l)}, \qquad a^{(l)} = \sigma\left(z^{(l)}\right).$$

2. *Output error.* For a differentiable output activation, compute

$$\delta^{(L)} = \sigma'\left(z^{(L)}\right) \odot \frac{\partial C}{\partial a^{(L)}}.$$

For softmax with cross-entropy, use

$$\delta^{(L)} = \frac{a^{(L)} - y}{n}.$$

3. *Backward pass.* For $l = L-1, L-2, \ldots, 1$, compute

$$\delta^{(l)} = \left(\delta^{(l+1)} (W^{(l+1)})^\top\right) \odot \sigma'\left(z^{(l)}\right).$$

4. *Gradients.* Form $\frac{\partial C}{\partial W^{(l)}} = (a^{(l-1)})^\top \delta^{(l)}$ and $\frac{\partial C}{\partial b^{(l)}} = \sum_{i=1}^{n} \delta_i^{(l)}$.
5. *Update.* With stochastic gradient descent and learning rate γ, update the weights and biases:

$$W^{(l)} \leftarrow W^{(l)} - \gamma (a^{(l-1)})^\top \delta^{(l)}, \qquad b^{(l)} \leftarrow b^{(l)} - \gamma \sum_{i=1}^{n} \delta_i^{(l)}.$$

(Hjorth-Jensen, 2016, p. 350)

1. Backpropagation

The backpropagation is the most important part of the neural network, as it is this process that creates the gradients of each layer, which are used to update the weights and biases. This process is different depending on the cost function, since the derivative of the cost function is the value we want to minimize. Thus it is different for the regression problem, which uses mean squared error, and the classification which uses cross entropy. The input layers, the z values and the output values a is generated

by the feed forward pass saver, the backpropagation iterates backwards, from the last layer to the first. Regressor, mean squared error:

$$\text{Output layer:} \quad \frac{\partial C}{\partial a^{(L)}} = \text{cost function derivative}, \tag{16}$$

$$\text{Hidden layer:} \quad \frac{\partial C}{\partial a^{(l)}} = \delta^{(l+1)} \left(W^{(l+1)}\right)^{\top}, \tag{17}$$

$$\delta^{(l)} = \frac{\partial C}{\partial z^{(l)}} = \frac{\partial C}{\partial a^{(l)}} \odot \sigma'\left(z^{(l)}\right), \tag{18}$$

$$\frac{\partial C}{\partial W^{(l)}} = \left(a^{(l-1)}\right)^{\top} \delta^{(l)}, \tag{19}$$

$$\frac{\partial C}{\partial b^{(l)}} = \sum_{i=1}^{n} \delta_i^{(l)}, \tag{20}$$

$$\nabla^{(l)} = \left(\frac{\partial C}{\partial W^{(l)}}, \frac{\partial C}{\partial b^{(l)}}\right). \tag{21}$$

Cross entropy, classification.

$$\text{Output layer:} \quad \Delta^{(L)} = \frac{\partial C}{\partial Z^{(L)}} = \frac{A^{(L)} - Y}{n}, \tag{22}$$

$$\text{Hidden layers:} \quad \begin{aligned} \Delta^{(l)} &= \frac{\partial C}{\partial Z^{(l)}} \\ &= \left(\Delta^{(l+1)} (W^{(l+1)})^{\top}\right) \\ &\quad \odot \sigma'\left(Z^{(l)}\right), \\ &\quad l = L-1, \ldots, 1. \end{aligned} \tag{23}$$

C. Stochastic Gradient Descent

The stochastic gradient descent function trains the neural network over a specified number of epochs. One epoch is one pass through the training data. At the start of each epoch, the data are shuffled and divided into mini-batches, each containing up to `batch_size` examples. Thus, for N training examples and batch size B, each epoch contains $\lceil N/B \rceil$ mini-batches. The network is updated after each mini-batch, using gradients calculated from that batch with backpropagfation. This is called mini-batch stochastic gradient descent. In the implementation, the backpropagation alghorithm calculates the mini-batch gradients, and the selected optimizer, plain, adam or rmsprop, updates the weights and biases. L1 or L2 penalty terms can also be added to the weight gradients, corresponding to Lasso and Ridge. The optimizer is selected separately from the mini-batch training procedure.

D. Neuralnetregressor

This estimator wraps the network in a scikit-learn-compatible interface. Its `fit` method creates the network using the requested input size and hidden-layer architecture. It selects the hidden activation function and its derivative, uses the identity function at the output layer, and supplies the mean squared error cost and its derivative.

Training is delegated to `train_sgd` or `train_plain`, according to the `descent` parameter. For SGD, the `method` parameter selects plain gradient descent, Adam, or RMSprop. The training function updates the weights and biases; the fitted network and training history are retained by the regressor.

The `predict` method passes new inputs through the fitted network and returns a one-dimensional NumPy array with one continuous prediction per example.

E. Implementation and analysis of Regressor

1. Model verification

The neuralnetregressor is implemented as an object, and verified against manually creating a neural network object and using stochastic gradient descent to train the model, both objects predict on the same data and the mean squared error are calculated on the same data, to ensure they agree. The gradients calculated by the backpropagation algorithm, are valified by calculating the gradient on the same data by automatic differentiation using jax. Jax is a verified solution used by scientists and data analysists, thus this verification ensures that the backpropagation calculates the correct gradients.

a. Gradient and training validation We tested a network with one hidden layer of 50 nodes and a network with two hidden layers of 50 and 100 nodes. Both used sigmoid hidden-layer activations. For all three training methods, the manually calculated gradients matched those from `jax.grad` according to `np.allclose`. The regressor and manual SGD implementations also produced matching predictions.

TABLE II Gradient validation and SGD training MSE for a network with two hidden layers containing 50 and 100 nodes, respectively.

Method	Gradient val.	Regressor val.	SGD train MSE
Plain	Passed	Passed	0.1084
Adam	Passed	Passed	0.01873
RMSprop	Passed	Passed	0.01394

TABLE I Gradient validation and training MSE for a network with one hidden layer of 50 nodes.

Method	Gradient val.	Regressor val.	SGD train MSE
Plain	Passed	Passed	0.1031
Adam	Passed	Passed	0.02404
RMSprop	Passed	Passed	0.02142

The gradient validation passed for all methods, and both the regressor and the manual use of sgd agreed. Adam and RMSprop achieved lower training MSE than plain stochastic gradient descent. The code can be found at (TimHHolmen, 2026b)

F. Model selection

1. Skicit Learn

Scikit-learn has multiple classes that can be used to validate and select between different models. The following classes have been utilised in the analysis of the Neuralnetwork regressor,

1. **KFold**: Creates k folds with shuffled values, k is set to 5 in this project. With random state = 2026.
2. **make$_\mathbf{p}$ipeline**: Creates a pipeline with scikit's StandardScaler() from the training data, with the NeuralNetRegressor as a chosen model.
3. **cross$_\mathbf{v}$al$_\mathbf{s}$core**: Splits the input data and target values, using the Kfold splitter, fits the pipeline on the training portion of each split, and evaluates it on the validation set of the split. Returns one score per split, with standard deviation.
4. **cross$_\mathbf{v}$alidate**: Does the same operation as cross val score, returns the training score in addition to the validation score for each fold, with standard deviation. Both classes should return the same validation score.
5. **GridSearchCV**: The GridSearchCV is the most extensively used class in this project. It uses the same Kfold class and pipeline as the previous cross validation classes. It has an additional $parameter_grid$ input, which uses the input values of the neuralnetregressor as variable model parameters, this allows for models selection on different parameters, such as the learning rate, number of hidden layers and gradient descent methods, plain, adam and rmsprop. This class creates and stores models for all chosen parameters within the grid, and fits on the chosen data using the regressor, and returns the cross validation scores on the training and validation set, the chosen score for the regressor is the negative mean squared error.

2. Cross Validation

The first **make$_\mathbf{p}$ipeline** is created with the NeuralNetRegressor class with one hidden layer of 50 nodes, sigmoid as activaiton function of the hidden layer, a learning rate of 0.01 with plain gradient descent as the standard method, the epoch is set to 100 with stochastic gradient descent with a batch size of 10. The seed is set to 2026. **KFold** is set with 5 folds, shuffle and random state with seed 2026. Both cross validation methods on this particular regressor model, is chosen with negative mean squared error as the scoring value.

3. GridSearchCV

The gridsearch cross validation is done using the chosen Kfold and neural network regression class as input variables. The cross validation is done with a different parameter grid and GridSearchCV model for each step, after each step the best model parameters within one standard error of the mean squared error is chosen, and used in the next neural regressor model, which in turn is the chosen model for the new GridSearchCV model. GridSearchCV stores the best model and its scorer, the best model corresponds to the lowest validation mean squared error of all the models. The first parameter grid is. [htbp] Best model parameters

TABLE III Parameter grid 1

Parameter	Values				
Gamma	0.001	0.003	0.01	0.03	0.01
Method	rmsprop	adam	plain		
Hidden	(50,)	(50, 100)			

within ONE-SE is found through analysing the results of the GridSearchCV, the best model parameters gamma $-learningrate$, method and hidden layers are chosen for the next neuralnetregressor model input of the next GridSearch. The second parameter grid is. [htbp] [htbp] Here

TABLE IV Parameter grid 2

Parameter	Values			
Activation	sigmoid	relu		
Hidden	(50,)	(30,)	(50, 100,)	(50, 100, 100,)

TABLE V Parameter grid 3

Parameter	Values			
Penalty	None	L1	L2	
Lambda	0.0001	0.001	0.01	0.1
Gamma	0.003	0.01	0.03	0.1

lambda equals the value of the penalty, L1 and L2 are the norms. The last parameter grid is split in half, where

lambda is set to zero when the penalty is None, else the lambda values are as shown in the table.

a. *One SE* It is important to specify what is meant by a one se model in this project. When a model is chosen during cross validation, based on the lowest mean squared error, if another model has a mse is within one standard error of the lowest, the choice between these two models would be arbitrary if mse was the only qualification, since one cannot tell, which model actually has the lowest mse, because the standard error reaches around both models mse. Thus, between these two models the least complex, most efficient model is chosen, this is also true if more than two models are within one se of the best. What model to choose then within one se of the best model, is then dependent on the model and the parameters which are varied during the cross validation.

4. OLS, Ridge and Lasso

G. Final test

H. Neuralnetclassifier

I. Neuralnetclassifier

J. Measure of quality

1. Confusion matrix

IV. RESULTS

V. DISCUSSION

VI. CONCLUSION

Appendix A: Use of AI/LLM Tools

1. Tools used

List every LLM tool used for the project, including its model or version, how it was accessed, and the approximate dates of use.

- *[Tool/version; access method; date used]*
- *GPT 6-Luna (codex), used 22.10–06.11.2026*
- *[GPT 6-Luna/version; GitHub Copilot (coding assistant session);* 22.10 – 06.11.2026*]*

2. Text contributions

For each report section, enter the highest applicable contribution level and briefly describe the assistance. Use level 0 for no assistance, level 1 for language-only edits, level 2 for editorial changes, and level 3 for substantial drafting. Replace the placeholders with an accurate record.

Abstract:: Level *[0–3]*; *[Describe assistance, or write "none"]*

Introduction:: Level *[0–3]*; *[Describe assistance, or write "none"]*

Theory:: Level *[0–3]*; *[Describe assistance, or write "none"]*

Methods:: Level *[0–3]*; *[Describe assistance, or write "none"]*

Results:: Level *[0–3]*; *[Describe assistance, or write "none"]*

Discussion:: Level *[0–3]*; *[Describe assistance, or write "none"]*

Conclusion:: Level *[0–3]*; *[Describe assistance, or write "none"]*

Level 3 prompt and revision note (delete if not applicable): [Summarise the prompt or task, how the generated text was changed, and how it was checked.]

3. Code contributions

List each code file or notebook included in the submission. Use level 0 for no assistance, level 1 for debugging, level 2 for a snippet, level 3 for a skeleton, and level 4 for substantial generation. Include what was tested, adapted, or verified.

`[Path / filename]`**::** Level *[0–4]*; *[Describe assistance and your testing/adaptations]*

Duplicate the code entry above for every submitted code file or notebook.
For every LLM-assisted function at level 2 or above, add an `LLM-assisted` note to its docstring. Add an inline note for assisted code blocks shorter than a function, and a Markdown note above assisted notebook cells.

a. LLM Declaration this section:

The skeleton of this section was created using GitHub Copilot. (modelversion unknown; coding-assistant session, November 8, 2026).

REFERENCES

M. Hjorth-Jensen (2016), *Machine Learning and the Physical Sciences* (Springer).

OpenAI (????), "Codex,".

TimHHolmen (2026a), "Fys-stk4155 project 2 repository," https://github.com/Timholmen96/FYS-STK4155/tree/main/Prosjekt2.

TimHHolmen (2026b), "Project 2 test regressor notebook," https://github.com/Timholmen96/FYS-STK4155/blob/main/Prosjekt2/Prosjekt-Kode/Project2_Test_Regressor.ipynb.